ChatGPT

Comparison of mmPoint and ZS-CIR Repositories

mmPoint (Dense Human Point Cloud from mmWave) – Contents & Reproducibility

Code Completeness: The mmPoint GitHub repo provides **full source code** implementing the model's major components. Specifically, it includes the **Multi-Modal Encoder (MME)**, the **Lift-and-Deform Modules (LDM)**, and the **Multi-Resolution Decoder (MRD)** architecture as described in the paper. For example, the `Model` class instantiates a radar encoder (`HuPRNet`) and three sequential **Lift** and **Deform** module pairs (upsampling the point cloud from 256 → 512 → 1024 → 2048 points) [1] [2]. The code for `Lift` and `Deform` modules is present under `models/utils/decoder.py`, implementing the point upsampling ("lift") and refinement ("deform") steps. The radar encoder (`HuPRNet`) is defined (reusing layers from the HuPR project) to process mmWave input volumes and output a 1000-dim feature, which is then reduced to 128-dim for conditioning the decoder [3]. In short, **all parts of the mmPoint model architecture (MME, LDMs, MRD)** appear to be included in the repository.

Data Handling and Scripts: The repository does not bundle the actual dataset but provides clear steps to obtain and prepare it. According to the README, users must retrieve: (1) the raw mmWave radar dataset from the HuPR project, and (2) the "pseudo" dense 3D human point clouds generated by the authors as ground truth [4]. A Google Drive link is given for ~34,800 point cloud files (58 scenes × 600 frames each, with 2048 points per frame) [5]. The code includes **preprocessing tools** (`tools/bin2npy.py` and `tools/radarnpy2hdf5.py`) to convert raw radar sensor `.bin` files into NumPy arrays and then into HDF5 files for training [6]. For loading data during training, a `HuPRDataset` class is provided (`dataset/dataset_hupr_npy.py`), which reads the radar `.npy` and corresponding point cloud `.npy` for each frame, normalizes the point cloud, and returns both the input radar tensor and target points [7] [8]. This means **dataset loading and preprocessing code is included**, though the actual data must be downloaded separately.

Training & Evaluation Scripts: The repo contains a top-level **training script** (`train.py`) and a **prediction script** (`tools/predict.py`). The training script uses a YAML config (e.g. `cfgs/mmPoint.yaml`) to set parameters like dataset paths and hyperparameters, then builds the dataset and model to run training (with PyTorch) [9]. The prediction script allows inference with a trained model: it loads a given checkpoint, a sample radar `.npy` file, and an initial 256-point human template (provided under `human_template/`), then produces a dense point cloud output [10] [11]. The README provides usage instructions (e.g. how to run training with `python train.py -c cfgs/mmPoint.yaml` and how to run prediction to generate output point clouds) [12] [13]. This indicates the **training and testing pipeline is documented and implemented**. (Note: There isn't a dedicated evaluation script for computing metrics; presumably one would need to compute Chamfer/EMD metrics from the outputs manually or from training logs.)

Pretrained Model Availability: No pretrained checkpoints are provided in the repo. The authors did not publish a model release on GitHub (the releases section is empty) [14], nor do they link to a pretrained `.pth` in the README. The expectation is that users will train the model from scratch using the provided

code and dataset. The `predict.py` script explicitly asks the user to specify the path to a "trained model .pth file" (their own checkpoint) before running inference [15]. Thus, to reproduce the exact paper results, one must go through the training process unless the authors share weights elsewhere. In summary, **all necessary code is present to train and run mmPoint, but you must obtain data and train the model (since no pretrained weights are included)**.

Documentation & Dependencies: The README gives concise setup guidance. It lists required environment: Python 3.6, PyTorch 1.10.2, CUDA 11.3, and includes a `requirements.txt`. Notably, mmPoint relies on custom CUDA ops (Chamfer distance, EMD, PointNet++ layers), and the installation steps cover compiling these modules [16]. This adds some complexity (the user must successfully build these extensions). Other than that, the code is fairly self-contained. The README credits the HuPR repository and indeed much of the radar processing code is borrowed from HuPR [17]. Overall, the documentation is sufficient to **guide reproduction**: it explains dataset preparation, installation, training, and inference. A motivated user can follow these steps to reproduce the results, given the hardware and time to train.

What Might Be Missing or Challenging: The mmPoint repo does not include the **raw data or ground-truth point clouds** (only links are provided), which is expected due to size. One potential gap is the lack of example configuration files or explicit evaluation scripts for measuring performance on a test set – users may need to extract metrics on their own. Also, without a pretrained model, reproducing results means incurring the full training process (which could be time-consuming). Nonetheless, **no critical code components appear missing**: the network, data pipeline, and training loop are present. The main hurdle is simply acquiring the data and training the model. Adapting mmPoint to a *novel application* would require similar data (mmWave inputs with corresponding point cloud outputs). If a student wanted to extend it, they might need access to mmWave sensors or alternative radar datasets, as the model is quite specialized to this input domain.

ZS-CIR (Zero-Shot Composed Text-Image Retrieval) – Contents & Reproducibility

Code Completeness: The ZS-CIR repository is a full PyTorch implementation of the approach (dubbed "TransAgg" in the paper). It appears very complete and self-contained, incorporating all necessary components for training and evaluation of composed image retrieval. The repo is organized with a `model/` directory that includes the model definitions – notably, it vendors the **CLIP** and **BLIP** model code inside (e.g. `model/clip/` and `model/BLIP/` subfolders). This means the code includes the vision-language backbone models needed for the method, rather than relying on external libraries at runtime. The integration of these large pre-trained models is explicitly acknowledged [18]. There are also custom model parts (likely the transformer aggregator "TransAgg" logic) in the `model` folder, but overall no critical pieces seem absent. The repository also has `data/` scripts to handle different datasets (e.g. `fiq_dataset.py` for FashionIQ, `cirr_dataset.py` for CIRR, and scripts for their LAION-derived data). In summary, **the codebase covers all necessary model and data-loading code**, and it's structured for multi-dataset training.

Dataset Accessibility: ZS-CIR tackles multiple datasets, and the README clearly lists how to obtain them. It references three custom composed-image datasets derived from LAION (Template, LLM, Combined) with a Google Drive link provided [19]. It also directs users to the public **FashionIQ** and **CIRR** datasets via their official repositories [20]. While these datasets are external, they are standard in the research community.

The need to download a large amount of data (LAION-based sets) is noted – e.g. issue #8 in the repo discusses a missing metadata file for a 600M image subset, implying very large scale data is involved. However, the authors mitigated this by sharing the processed datasets on Drive (presumably in a manageable form). Compared to mmPoint, **ZS-CIR's data are more broadly accessible** – they are image/text datasets, not requiring special sensors. A student can obtain FashionIQ and CIRR easily (they are available online), and use the provided LAION subset link for the training data.

Pretrained Models & Checkpoints: A strong advantage of ZS-CIR is that it provides **pretrained weights and model checkpoints**, making it easier to reproduce results quickly. The README's "Model Zoo" section gives links to required pretrained models: CLIP ViT-B/32, CLIP ViT-L/14, and BLIP (these are downloaded from official sources) [21]. In addition, the authors supply a Google Drive folder of **trained checkpoints** for their TransAgg model [22]. With these in hand, one can directly run inference or evaluation without retraining from scratch. (The GitHub releases section has none published, similar to mmPoint [23], but the Drive link serves the purpose.) The presence of checkpoints greatly lowers the barrier to reproducing the paper's quantitative results – you can load the model and test it on CIRR or FashionIQ as the paper did. By contrast, mmPoint had no such weights provided.

Training and Evaluation Procedures: The repository includes a top-level `main.py` for training and a specialized script for evaluation on the **CIRR** benchmark (`cirr_test_submission.py`). Training usage is straightforward: after adjusting any parameters in `config.py`, simply run `CUDA_VISIBLE_DEVICES=0 python main.py` [24]. The code will train the model on the configured dataset(s). For evaluation, the example given is for CIRR: `python cirr_test_submission.py` produces a results file in the format needed for the CIRR challenge [25]. There are likely similar evaluation pathways for FashionIQ (possibly via the training script or a notebook). The instructions are concise but clear. Additionally, the config system (using `config.py` and a simple argument structure) makes it relatively easy to switch datasets or tweak hyperparameters [24]. Overall, **the procedures to train and test are well-documented**. The code's dependency on large models (CLIP/BLIP) is handled by providing download links and including the code, so there are no unusual installation steps beyond standard Python packages (listed in `requirements.txt`).

Extending or Reusing the Project: ZS-CIR's code appears **highly usable and modular**, which benefits a student looking to propose a novel application. Because it builds on widely-used architectures (CLIP for image/text embedding, BLIP for captioning), one could envision extensions like swapping in new backbone models, fine-tuning on a new image-text dataset, or applying the composition retrieval idea to a different domain (e.g. zero-shot retrieval on a custom image collection) with relative ease. The dataset handlers and training loop can be adapted to other data as long as the format is similar (reference image + text → target image). There is also only minimal need for external proprietary data – all datasets cited are publicly available. The main "external dependencies" are the pretrained CLIP/BLIP weights, which are provided via public links [21] (no special hardware or sensor data needed). In short, for a student project, **ZS-CIR is easier to set up and alter**, since it leverages mainstream data and models.

Which Project is More Suitable to Reproduce and Extend?

Taking all factors into account, **ZS-CIR is generally more straightforward to implement and extend than mmPoint** for most students:

- **Code Usability:** Both repos have complete code, but ZS-CIR includes ready-to-use pretrained models and a unified PyTorch training pipeline that's familiar to anyone in vision-language research. mmPoint's code is also complete but requires compiling CUDA ops and careful setup of an older environment (PyTorch 1.10 on Python 3.6) [26]. Students might find ZS-CIR's code (mostly Python and some Jupyter notebooks) more approachable, whereas mmPoint involves C++/CUDA components (17.5% CUDA, 4.9% C++ by LOC) [27].

- **Dataset Accessibility:** ZS-CIR uses image/text datasets that are **public and widely-used** (LAION, FashionIQ, CIRR). These can be downloaded with an internet connection and don't require special hardware. mmPoint relies on a **specialized radar dataset** (HuPR) and the authors' generated point clouds. Accessing and handling this data is more niche – one must download large radar recordings and work in 5D radar data space, which is unfamiliar to most and possibly challenging to debug. If the student does not already work in the mmWave domain, obtaining and understanding the data could be a hurdle. By contrast, working with images and captions (as in ZS-CIR) is more conventional.

- **Clarity of Process:** Both projects have instructive READMEs, but ZS-CIR's instructions might lead to results faster. With ZS-CIR, you can download the provided checkpoint and immediately run the test script on a benchmark to see if you match the reported accuracy [28] [25]. This quick verification is invaluable. With mmPoint, reproducing results means first replicating the entire training – a longer process with more room for error. Also, mmPoint's outcome (3D point clouds) may require additional steps (e.g. visualization in MeshLab as suggested) to interpret [29], whereas ZS-CIR's outcome (retrieved images) can be directly evaluated with provided metrics or visually.

- **Extensibility and Novel Applications:** ZS-CIR's domain (composed image retrieval) is broad and can be extended without new kinds of sensors – a student could, for example, apply the model to a different image dataset or incorporate textual modifications in another language, etc., using just software and data. The **feasibility of proposing a novel application** is higher because the barriers are lower (one could use existing image databases or slightly tweak the model to a new multimodal task). On the other hand, extending mmPoint would likely mean venturing further into mmWave sensing. A novel application might be, say, using the mmPoint network for *other* radar scenarios or for integrating with other sensor modalities – but any such idea would still require collecting appropriate radar data or leveraging the specific output format of mmPoint. This is a narrower path, usually requiring domain expertise or hardware access that many students don't have. Without external dependencies, it's hard to repurpose mmPoint to something outside its original scope (since it is **highly specialized to radar-to-point-cloud translation**).

Overall Recommendation: For a student project, **ZS-CIR is the more suitable choice to reproduce and build upon**. Its codebase is complete, well-documented, and comes with pretrained models, making it easier to get running and verify against the paper. The datasets and problem domain are accessible, and the training/evaluation procedures are clearly laid out. mmPoint, while providing all necessary code to replicate the paper, demands a bigger upfront investment (data prep, lengthy training) and is tied to a niche modality (mmWave) which might limit straightforward extensions. In summary, mmPoint does contain

sufficient material to reproduce its BMVC paper (given the data and training time) [30] [4] and one could imagine applications in the radar/3D vision space, but **ZS-CIR's project is easier to implement and extend** due to its plug-and-play code, available models/data, and more ubiquitous application area.

Sources:

- mmPoint BMVC2023 code repository (Q. Xie *et al.*) [4] [1]
- ZS-CIR BMVC2023 code repository (Y. Liu *et al.*) [19] [21]

---

[1] [2] [3] mmPoint.py
https://github.com/NUAAXQ/mmPoint/blob/2219cfcc01892c965e09e4df057c2f8f8d06ffec/models/mmPoint.py

[4] [5] [6] [9] [12] [13] [14] [16] [17] [26] [27] [29] [30] GitHub - NUAAXQ/mmPoint: [BMVC 2023] mmPoint: Dense Human Point Cloud Generation from mmWave
https://github.com/NUAAXQ/mmPoint

[7] [8] dataset_hupr_npy.py
https://github.com/NUAAXQ/mmPoint/blob/2219cfcc01892c965e09e4df057c2f8f8d06ffec/dataset/dataset_hupr_npy.py

[10] [11] [15] predict.py
https://github.com/NUAAXQ/mmPoint/blob/2219cfcc01892c965e09e4df057c2f8f8d06ffec/tools/predict.py

[18] [19] [20] [21] [22] [23] [24] [25] [28] GitHub - Code-kunkun/ZS-CIR: [BMVC 2023] Zero-shot Composed Text-Image Retrieval
https://github.com/Code-kunkun/ZS-CIR